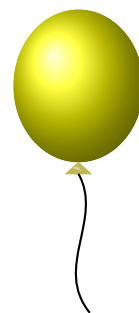


B Tirando del hilo



Cuando escribes algo en la barra del navegador, en la app de contactos, en el buscador del móvil, casi siempre aparece debajo una lista corta de sugerencias que completan lo que llevas escrito. La lógica es siempre la misma: tienes detrás un diccionario (direcciones visitadas, contactos guardados, productos del catálogo, términos del buscador) y, según vas tecleando, el sistema te enseña las entradas que empiezan por lo que llevas escrito. Pero no te las enseña todas: por una cuestión de espacio en pantalla y de no marear al usuario, el sistema fija un tope K y te muestra como mucho K sugerencias por consulta. Si hubiera más, el resto se quedan ocultas y solo afloran cuando escribes alguna letra más y reduces el grupo.

A Marta, que trabaja en una empresa que desarrolla este tipo de servicios, le ha caído un encargo curioso. Un cliente tiene ya en producción un autocompletado que funciona, pero se ha quedado corto: es lento, no escala, y los algoritmos de ordenación de sugerencias son los que vienen de serie. Quieren que el equipo de Marta les construya una versión nueva, más rápida y configurable.

El problema es que Marta necesita el diccionario completo de entradas que el sistema actual está sirviendo. Y resulta que nadie sabe ya dónde está: el desarrollador original se marchó hace años, el código fuente está perdido en algún servidor que ya nadie mantiene, y lo único que queda en pie es el servicio en producción, escuchando peticiones y devolviendo sugerencias como el primer día. Marta puede utilizarlo escribiendo el prefijo que quiera y el sistema le devolverá, obedientemente, las hasta K entradas del diccionario que empiezan por ese prefijo. Si hubiera más, solo verá las primeras en orden lexicográfico. Para reconstruir el diccionario completo no le queda más remedio que ir tirando del hilo a base de preguntas.

INTERACCIÓN

Este es un problema *interactivo*. Tu programa intercambiará mensajes con el juez según las reglas que se describen a continuación.

Primero, el juez escribe en la primera línea un entero T ($0 \leq T \leq 100$), el número de diccionarios que tendrás que reconstruir en esta ejecución. A continuación, se procesan los T casos uno tras otro. Por cada caso, el juez comienza escribiendo una línea con dos enteros K y L separados por un espacio ($2 \leq K \leq 20$, $1 \leq L \leq 8$): K es el número máximo de palabras que el juez devolverá en cada respuesta, y L es la longitud máxima de cualquier palabra del diccionario. Las palabras del diccionario son cadenas no vacías de letras minúsculas inglesas, todas distintas, de longitud entre 1 y L . La suma de los números de palabras de los T diccionarios es a lo sumo 500.

A partir de ese momento, tu programa puede emitir dos tipos de mensajes:

- **Consulta:** $? \text{pre}$, donde pre es un prefijo formado únicamente por letras minúsculas inglesas (a-z), de longitud variable. Si el prefijo es vacío, basta con escribir $?$ (sin espacio detrás).

El juez responde con una línea que contiene un entero m ($0 \leq m \leq K$), seguida de m líneas con una palabra cada una. Las palabras devueltas son las m palabras lexicográficamente menores del diccionario que empiezan por **pre**, listadas en orden ascendente.

- **Respuesta:** **! n**, donde n es el número de palabras del diccionario, seguido de n líneas con las palabras del diccionario en orden lexicográfico estrictamente ascendente. Esta línea cierra el caso y el juez pasará al siguiente (o terminará la ejecución si era el último).

Para reconstruir cada diccionario puedes hacer **como mucho** $4 \cdot 10^4$ **consultas**. Si te pasas, el juez abortará la conversación: en lugar de responder con un entero m a tu siguiente consulta te enviará la línea **-1**. En cuanto recibas **-1** debes terminar tu programa de inmediato con código de salida 0. Tu envío recibirá veredicto incorrecto.

Al tratarse de un problema *interactivo* es importante que cada vez que tu solución escriba algo que el juez deba leer se asegure de volcar la salida (usando terminología inglesa, haga *flush*). La forma de hacerlo varía entre lenguajes. En los admitidos en este concurso puede hacerse con:

- C++: `cout << endl;` o `cout << flush;`.
- C: `fflush(stdout);`.
- Java: `System.out.flush();`.
- Python: `print(..., flush=True)` o `sys.stdout.flush()`.

EJEMPLO DE INTERACCIÓN

A modo de ilustración, supongamos un único caso ($T = 1$) con $K = 2$ y $L = 7$. La siguiente tabla muestra un intercambio plausible: en la primera columna, las líneas que escribe el juez; en la segunda, las que escribe el programa. Las filas se leen de arriba abajo en orden cronológico.

juez	programa
1	
2 7	
	? cas
2	
casa	
caserio	
	? cor
1	
coral	
	? d
0	
	? a
1	
asa	
	! 4
	asa
	casa
	caserio
	coral

Las consultas mostradas no bastan para garantizar la respuesta final ni reflejan ninguna estrategia; solo ilustran el formato del intercambio.